



**Universidad Tecnológica Nacional
Facultad Regional Córdoba**

Algoritmos y Estructuras de Datos

Ficha Nro. 16

Arreglos Bidimensionales

Prof. Ing. Corso Cynthia

Temario

- Introducción
- Matrices (arreglos bidimensionales)
 - Creación
 - Carga de elementos de la matriz
 - Acceso de elementos
 - Recorridos. Por filas y por columnas
 - Sumatoria de elementos de la matriz por columna.

Introducción

- **Arreglo unidimensional:** es una estructura de datos que permite almacenar y procesar un conjunto significativo de datos.
- Los problemas analizados hasta ahora fueron de naturaleza unidimensional (cantidad de índices para acceder a los elementos) y estos se utilizaron como medio de soporte para almacenar los datos.
- Sin embargo, existen otras situaciones en los que se necesita organizar los datos de manera que el acceso a los datos es necesario mediante dos o más índices.

Horario de Clases

Curso: _____

Hora	Lunes	Martes	Miércoles	Jueves	Viernes

Segunda Edición

D186 02/07/2013

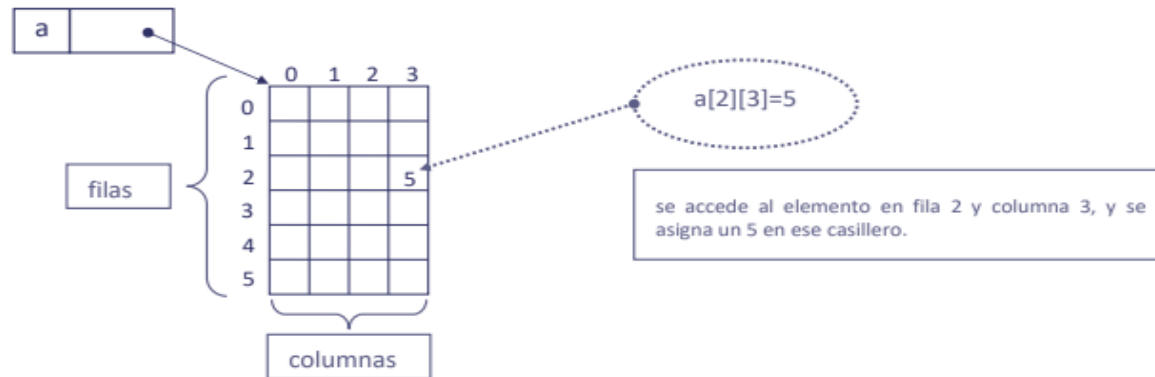
	A	B	C	D	E	G	H	I
1		Control GASTOS CASA						
2								
3								
4		Gasto	Importe	Fecha				
167		Gas	546,29 €	11/04/2013				
168		Teléfono	64,98 €	10/04/2013				
169		Comunidad	247,95 €	03/04/2013				
170		Comunidad Garaje	37,51 €	03/04/2013				
171		Guardería	580,69 €	09/04/2013				
172		Teléfono	66,50 €	13/05/2013				
173		Comunidad	247,95 €	06/05/2013				
174		Comunidad Garaje	37,51 €	06/05/2013				
175		TS Coche	314,24 €	06/05/2013				
176		TS Coche	419,51 €	06/05/2013				
177		Guardería	580,69 €	08/05/2013				
178		Luz	264,70 €	21/06/2013				
179		Gas	309,36 €	19/06/2013				
180		Teléfono	68,89 €	10/06/2013				
181		IBI	291,27 €	05/06/2013				
182		IBI Garaje	23,61 €	05/06/2013				
183		Agua	84,17 €	24/06/2013				
184		Seguro Casa	575,58 €	17/06/2013				
185		Comunidad	247,95 €	02/07/2013				
186		Comunidad Garaje	37,51 €	02/07/2013				
187		Total	42.113,81 €					
188								

Para añadir gastos, haz clic en la última fila de registros y pulsa la tecla Tabulador.

Matrices

- **Concepto:** es un arreglo bidimensional cuyos elementos están almacenados en un formato de tabla conformado por:
 - filas (disposiciones horizontales) y
 - columnas (disposiciones verticales).
- Para el **acceso** de cada elemento es necesario el uso de **dos índices**.

Figura 1: Esquema de representación de una matriz de orden 6*4.



```
a = [ [2, 3, 4, 5],
      [1, 5, 6, 3],
      [7, 2, 4, 0],
      [8, 4, 1, 7],
      [0, 0, 2, 9],
      [1, 3, 4, 1],
    ]
```

```
a[2][3] = 5 # accede a una casilla y cambia su valor.
```

Matrices: forma de implementación en Python

- En Python las matrices se implementan con el tipo list, usando el concepto de “**listas embebidas**”, ya que una list puede contener otras “list” embebidas en ella.
- Es decir que se implementan listas de listas (variable de tipo list, que en cada celda tiene a otra list).



Matrices: Creación (i)

Creación de la matriz con **asignación directa de valores constantes**.
Existen dos posibilidades:

- Posibilidad 1: indentación de cada fila:

```
m0 = [ [1, 3, 4],  
        [3, 5, 2],  
        [4, 7, 1]  
      ]  
print('Matriz con valores fijos:', m0)
```

- Posibilidad 2: sin indentación

```
m1 = [ [1, 3, 4], [3, 5, 2], [4, 7, 1] ]  
print('Matriz con valores fijos:', m1)
```

Matrices: Creación (ii)

Creación de la matriz **sin valores fijos (constantes)**, con “n” filas y “m” columnas (Donde “n” y “m” son variables).

En Python las matrices son construidas en tiempo de ejecución. Estudiaremos tres alternativas posibles:

- **Alternativa 1:** la mas descriptiva e intuitiva, pero la mas ineficiente.
- **Alternativa 2:** menos detallada e intuitiva, pero mas compacta que la anterior.
- **Alternativa 3:** mas compacta y eficiente, aunque posiblemente menos clara.

Matrices: Creación (iii)

- **Alternativa 1:** Supongamos se desea crear una matriz denominada **m1** con “n” **filas** $\rightarrow n = 3$ y “m” **columnas** $\rightarrow m = 2$.

1) La idea es partir de una lista vacía, y asignar en ella “n” listas vacías.

```
m1 = []
for f in range(n):
    m1.append([])    m1=> [ [], [], [] ]
```

2) Agregar en cada una de las “n” listas de m1, un total “m” de elementos.

```
m1 = []
for f in range(n):
    m1.append([])
    for c in range(m):
        m1[f].append(None)
```

Apariencia final **m1** = [[None, None], [None, None], [None, None]]

Matrices: Creación (iv)

- **Alternativa 2:** Supongamos se desea crear una matriz denominada m2 con “n” **filas** → **n = 3** y “m” **columnas** → **m = 2**.

1) Se comienza creando “n” filas con el operador multiplicación.

```
m2 = [None] * n      => m2 = [ None, None, None ]
```

2) Para completar la matriz se itera sobre los elementos, , en este caso None, y se los reemplaza por una lista de “m” elementos.

```
m2= [None] * n
for f in range(n):
    # crea la columnas para cada fila
    m2[f] = [None] * m # Expande cada fila a 2 elem.
```

Apariencia final: m2 = [[None, None], [None, None], [None, None]]

Matrices: Creación (v)

- **Alternativa 3:** Supongamos se desea crear una matriz denominada `m3` con “n” **filas** $\rightarrow n = 3$ y “m” **columnas** $\rightarrow m = 2$. Para su creación se utiliza una **expresión de comprensión**.

```
m3 = [ [None] * m for f in range(n) ]
```

- El ciclo for realiza una iteración de “n” vueltas.
- En cada vuelta crea una lista de “m” elementos “None”.
- Si el valor de filas es $n = 2$ y columnas es $m = 3$ la estructura de la matriz generada sería:

Apariencia final: `m3 = [ [None, None], [None, None], [None, None] ]`

Matrices: Acceso Individual

- El **acceso individual** en una matriz es mediante dos índices:
 - El primero selecciona “la fila” (o sea una de las “n” sublistas) y
 - El segundo selecciona “la columna” (o sea el elemento dentro de la sublista seleccionada con el primer índice). Ejemplos:

```
matriz[0][3] = 10
```

```
matriz[1][2] = 20
```

- El **primer índice** de cada dimensión es **cero**.
- Los **índices de acceso** a filas y columnas **no puede** sobrepasar al espacio reservado.
- Si se dispone una matriz de dimensión 2 x 2, la siguiente operación sería incorrecta:

```
a = matriz[0][4] #Error
```

Índice que sobrepasa la dimensión reservada para las columnas...



Matrices: Carga de elementos

- Se usan dos ciclos “for” uno para recorrer las filas y el otro para las columnas. De esta forma se accede a una celda en particular.

```
fil, col = 3, 3
mat = [[None] * col for f in range(fil)]
for f in range(len(mat)):
    for c in range(len(mat[0])):
        mat[f][c] = int(input('Nro:'))
```

`len(mat)` → Retorna cantidad de “filas”

`len(mat[0])` → Retorna cantidad de “columnas”

Solicita al usuario por teclado un valor numérico y lo asigna en una casilla de una determinada “fila” y “columna”.

Ejemplo de matriz ya cargada.

	0	1	2
→ 0	1	2	3
→ 1	4	5	6
→ 2	7	8	9



En Python
`mat = [[1,2,3],[4,5,6],[7,8,9]]`

Matrices: Recorrido completo (i)

- Recorrido completo de la matriz por “filas”: se usan dos ciclos “for” uno para recorrer las filas y el otro para las columnas.

```
# Se supone ya creada una matriz
# denominada mat de orden n * m.
```

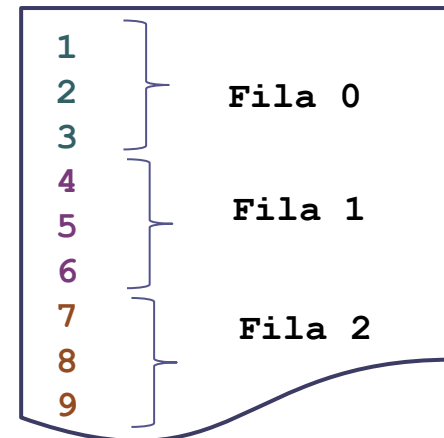
```
for f in range(len(mat)):
    for c in range(len(mat[0])):
        print(mat[f][c])
```

`len(mat)` → Retorna cantidad de “filas”

`len(mat[0])` → Retorna cantidad de “columnas”

Visualiza los elementos de la matriz por “filas”

	0	1	2
→ 0	1	2	3
→ 1	4	5	6
→ 2	7	8	9



Matrices: Recorrido completo (ii)

- **Recorrido completo de la matriz por “columnas”**: se deben invertir los dos ciclos “for”, llevar mas afuera el que recorre las columnas y mas interno el que recorre las filas.

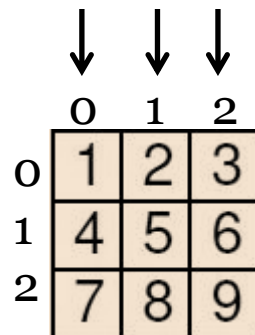
```
# Se supone ya creada una matriz
# denominada mat de orden n * m.
```

```
for c in range(len(mat[0])):
    for f in range(len(mat)):
        print(mat[f][c])
```

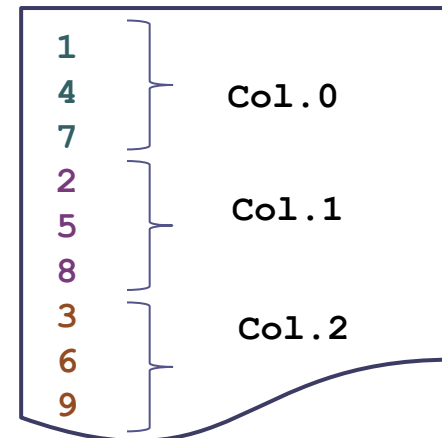
`len(mat[0])` → Retorna cantidad de “columnas”

`len(mat)` → Retorna cantidad de “filas”

Visualiza los elementos de la matriz por “columnas”



	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9



1	{	Col.0
4		
7		
2	{	Col.1
5		
8		
3	{	Col.2
6		
9		

Matrices: Recorrido completo (iii)

A tener en cuenta!!!

La variable que indica la **fila** va en el **primer par de corchetes**, y la variable que indica la **columna** va en el **segundo par de corchetes**, sin importar cuál ciclo va por fuera y cuál por dentro.



Matrices: Totalización de elementos por columnas

```
# Script de carga por teclado de la matriz "mat"
filas, columnas = 3, 3
mat = [[0] * columnas for f in range(filas)]
for c in range(columnas):
    for f in range(filas):
        mat[f][c] = int(input('Valor: '))
print('Matriz leída:', mat)
```

```
# Script para totalizar los elementos de la matriz "mat"
acu = 0
for c in range(columnas):
    for f in range(filas):
        acu+ = mat[f][c]
print('Acumulación de datos de mat:', acu)
```

	↓	↓	↓
	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
acu	12	27	45

Muchas Gracias por la atención!!

Cualquier duda pueden contactarme por mensajería interna del Aula Virtual.